

GitHub 협업 설명서

Game Stack 팀 프로젝트용

처음 하는 사람도 따라할 수 있는 단계별 가이드

이 문서를 읽고 나면 할 수 있는 것

- ✓ 코드를 GitHub에 올리고 받을 수 있다
- ✓ 브랜치를 만들어 내 작업공간을 분리할 수 있다
- ✓ 팀원과 코드를 합칠 수 있다
- ✓ 충돌 없이 협업할 수 있다

1장. 기본 용어 먼저 알기

Git과 GitHub를 처음 쓴다면 아래 용어들을 먼저 이해하세요. 이것만 알면 나머지는 자연스럽게 따라옵니다.

용어	쉬운 설명
Repository (레포지토리)	프로젝트 코드 전체를 담은 저장소입니다. Google Drive의 폴더와 비슷하지만 변경 이력까지 전부 기록됩니다.
Branch (브랜치)	작업 공간을 복사해 분리한 것입니다. RPG 게임의 '별도 슬롯에 저장'과 같습니다. 복사본에서 마음껏 작업하고, 완성되면 원본에 합칩니다.
Commit	작업 내용을 저장하는 행위입니다. 게임의 '세이브'와 같으며, 언제든지 이 시점으로 되돌아올 수 있습니다.
Push	내 컴퓨터의 코드를 GitHub(원격 서버)에 올리는 것입니다.
Pull	GitHub의 최신 코드를 내 컴퓨터로 받아오는 것입니다.
Clone	GitHub의 레포지토리 전체를 내 컴퓨터에 복사하는 것입니다. 처음 참여할 때 한 번만 합니다.
Merge	두 브랜치의 코드를 하나로 합치는 것입니다.
Pull Request (PR)	내 브랜치를 다른 브랜치에 합쳐달라고 요청하는 것입니다. GitHub 웹사이트에서 버튼 클릭으로 합니다.

2장. 브랜치 구조 이해하기

3명이 동시에 작업할 때 코드가 뒤섞이지 않도록 브랜치로 작업 공간을 분리합니다.

브랜치 종류

브랜치	역할	설명
main	최종 완성본	발표, 배포용. 완전히 완성된 코드만 올라옵니다. 직접 건드리지 않습니다.
dev	합치는 공간	각자의 기능이 완성되면 여기에 모입니다. 팀원들이 작업을 통합하는 브랜치입니다.
feat/auth	팀원 A 작업공간	로그인 기능을 맡은 팀원이 혼자 쓰는 브랜치입니다.
feat/search	팀원 B 작업공간	검색 기능을 맡은 팀원이 혼자 쓰는 브랜치입니다.
feat/library	팀원 C 작업공간	라이브러리 기능을 맡은 팀원이 혼자 쓰는 브랜치입니다.

핵심 원칙 feat/* 브랜치에서 작업한 뒤 → dev로 합치고 → 최종 완성 시 main으로 합칩니다.

절대로 main 브랜치에 직접 push하지 마세요. main은 항상 배포 가능한 상태여야 합니다.

3장. 최초 세팅 — 팀장만 하는 작업

팀장이 딱 한 번만 하는 작업입니다. 나머지 팀원은 이 챕터를 건너뛰고 4장으로 가세요.

3-1. GitHub에서 레포지토리 생성

1	github.com 접속 → 로그인
2	우측 상단 [+] 버튼 → [New repository] 클릭
3	Repository name에 <code>game-stack</code> 입력
4	Private 선택 (Public 선택 시 API 키 노출 위험!)
5	[Create repository] 클릭
6	Settings → Collaborators → [Add people] → 팀원 이메일 입력 → 초대
7	팀원들이 이메일로 받은 초대장을 수락하면 완료

3-2. 로컬에서 프로젝트 초기화

터미널(Mac: Terminal, Windows: Git Bash)을 열고 아래 명령어를 순서대로 실행합니다.

```
# Vite로 React 프로젝트 생성

npm create vite@latest gameStack -- --template react
cd gameStack
npm install

# Git 초기화 및 첫 커밋

git init
git add .
git commit -m "init: 프로젝트 초기 세팅"

# GitHub 레포지토리 연결 (주소는 본인 GitHub 주소로)

git remote add origin https://github.com/Jeonghun-k/gameStack.git
git push -u origin main

# dev 브랜치 생성 및 push

git checkout -b dev
git push origin dev
```

.gitignore 확인 필수 Vite 프로젝트에는 .gitignore가 자동 생성되지만, 반드시 .env가 포함되어 있는지 확인하세요.

4장. 초기 세팅 — 팀원이 하는 작업

팀장이 레포지토리를 만들고 초대를 보내면, 팀원은 아래 순서대로 세팅합니다. 처음 한 번만 하면 됩니다.

- 1 팀장에게 받은 GitHub 초대 이메일 → [Accept invitation] 클릭
- 2 터미널을 열고 작업할 폴더로 이동 (예: `cd ~/Desktop`)
- 3 레포지토리 복사: `git clone https://github.com/Jeonghun-k/gameStack.git`
- 4 폴더 진입: `cd game-stack`
- 5 패키지 설치: `npm install`
- 6 dev 브랜치로 이동: `git checkout dev`
- 7 .env 파일 생성 (팀장에게 카카오톡으로 받은 API 키 내용 붙여넣기)
- 8 서버 실행 확인: `npm run dev` → 브라우저에 화면이 뜨면 성공

.env 파일 작성 방법

프로젝트 루트(game-stack 폴더 바로 안)에 .env 파일을 직접 만들고 아래 내용을 입력합니다.

```
# .env 파일 내용 (팀장에게 받은 값으로 교체)

VITE_SUPABASE_URL=https://xxxxxxxxxxxxx.supabase.co
VITE_SUPABASE_ANON_KEY=eyJhbGciOiJIUzI1NiIsInR5cCI6I..
VITE_RAWG_KEY=rawg에서_발급받은_API_키
```

중요 .env 파일은 절대 git add 하지 마세요. .gitignore에 이미 등록되어 있어서 자동으로 제외되지만, 혹시라도 git add . 이후에 .env가 Staged에 잡히면 즉시 제외해야 합니다.

제외 명령어: `git rm --cached .env`

5장. 매일 작업 흐름 — 이것이 핵심

매일 작업을 시작할 때마다 아래 7단계를 반복합니다. 처음에는 어색하지만 3번만 해보면 몸에 익습니다.

STEP 1 최신 코드 받기

작업 시작 전 가장 먼저 해야 합니다. 팀원이 어제 올린 코드가 있을 수 있습니다.

```
git checkout dev          # dev 브랜치로 이동
git pull origin dev        # 최신 코드 받기
```

STEP 2 내 브랜치 만들기

내 작업공간을 새로 만듭니다. 이미 만들어둔 브랜치가 있으면 이 단계는 건너뜁니다.

```
git checkout -b feat/auth  # 로그인 담당이면
git checkout -b feat/search # 검색 담당이면
git checkout -b feat/library # 라이브러리 담당이면
```

이미 브랜치가 있을 때 -b 없이 git checkout feat/auth 처럼 이동만 하면 됩니다.

STEP 3 코드 작업

VS Code를 열고 맡은 기능을 개발합니다. 파일을 저장하고 브라우저에서 확인하면서 진행합니다.

STEP 4 변경사항 저장 (Commit)

작업이 어느 정도 마무리되면 저장합니다. 하루에 여러 번 해도 됩니다.

```
git status          # 변경된 파일 목록 확인
git add .           # 모든 변경사항 선택
git commit -m "feat: 로그인 기능 구현" # 메시지와 함께 저장
```

커밋 메시지 규칙

팀끼리 통일하면 나중에 변경 이력을 보기 쉽습니다.

접두사	언제	예시
feat:	새 기능 추가	<i>feat:</i> 게임 검색 기능 구현
fix:	버그 수정	<i>fix:</i> 검색 결과 안 뜨는 버그 수정
style:	CSS 수정	<i>style:</i> 게임 카드 디자인 수정
refactor:	코드 구조 개선	<i>refactor:</i> <i>useLibrary</i> 혹은 분리
chore:	설정 파일 수정	<i>chore:</i> <i>.env</i> 예시 파일 추가

STEP 5 GitHub에 올리기 (Push)

```
git push origin feat/auth # 본인 브랜치 이름으로
```

STEP 6 Pull Request 만들기

- 1 github.com → 본인 레포지토리 접속
- 2 상단에 노란 배너가 보이면 [Compare & pull request] 클릭 (안 보이면 [Pull requests] 탭 → [New pull request])
- 3 base를 **dev** 로, compare를 **feat/auth** (본인 브랜치)로 설정
- 4 제목 입력: feat: 로그인 기능 구현
- 5 카카오톡에 팀원들에게 Merge 해도 된다고 알리기
- 6 [Merge pull request] → [Confirm merge] 클릭

STEP 7 Merge 후 정리

```
git checkout dev      # dev로 돌아가기  
git pull origin dev    # 합쳐진 코드 받기
```

6장. 명령어 전체 요약

자주 쓰는 명령어를 한눈에 볼 수 있도록 정리했습니다. 헛갈릴 때 이 표를 참고하세요.

명령어	언제 사용	설명
<code>git pull origin dev</code>	작업 시작 전	dev 브랜치의 최신 코드를 내 컴퓨터로 받아옵니다
<code>git checkout -b feat/기능</code>	새 기능 시작 시	새 브랜치를 만들고 그 브랜치로 이동합니다
<code>git checkout dev</code>	브랜치 이동	dev 브랜치로 돌아갑니다
<code>git status</code>	언제든지	현재 변경된 파일 목록을 보여줍니다
<code>git add .</code>	커밋 전	변경된 파일 전체를 선택합니다
<code>git commit -m "메시지"</code>	add 후	선택한 파일들을 메시지와 함께 저장합니다
<code>git push origin feat/기능</code>	커밋 후	내 브랜치를 GitHub에 올립니다
<code>git log --oneline</code>	언제든지	커밋 이력을 한 줄씩 보여줍니다

7장. 자주 하는 실수와 해결법

실수 1 pull 없이 push해서 에러가 난 경우

```
# 에러 메시지 예시

! [rejected] main -> main (fetch first)

# 해결 방법

git pull origin dev # 먼저 최신 코드를 받고

git push origin feat/기능 # 다시 push
```

실수 2 .env 파일을 git add 해버린 경우

```
git rm --cached .env      # Staging에서 제거  
git commit -m "chore: .env 추적 제거"
```

실수 3 브랜치를 잘못 만든 경우

```
git checkout dev          # dev로 이동  
git branch -d 잘못된브랜치명  # 브랜치 삭제  
git checkout -b feat/올바른이름  # 다시 만들기
```

실수 4 어떤 브랜치에 있는지 모르는 경우

```
git branch      # 현재 브랜치 목록 확인 (* 표시가 현재 위치)
```

